# Task 1 — Build Your First Real Skill

**Abdul Manan** · Panaversity Agent Factory, AI-101 · Skill: `weekly-study-notes`

## PROJECT TITLE AND WHAT IT DOES

**weekly-study-notes** — a Skill that turns raw, messy class notes into an interactive single-file HTML study tool. One card per topic, five key terms per card that stay hidden until tapped, a live progress meter, and a scored three-question quiz at the end.

It takes pasted text or a Markdown file, invents its own topic headings (my raw notes never have any), cleans the grammar without adding facts, and outputs a self-contained artifact.

## WHY I CHOSE THIS SKILL AND HOW IT HELPS MY DAILY LIFE

I take notes during lectures the way most people do — fragments, typos, thoughts out of order. Every week I was re-typing the same instructions to get them cleaned up: split by topic, bold the key terms, give me questions to test myself. That is the definition of a task worth encoding once.

But the real reason is what happened when it worked. The first version produced correct, well-formatted Markdown — and I had no intention of ever re-reading it. **The skill was working and still useless.** Reading notes is passive; I skim and feel productive and retain nothing.

So the skill does not produce a document. It produces a study tool built on active recall: the key terms sit hidden, I have to say the definition out loud before tapping to check, and the quiz scores me. That mechanic is the whole point, and it is why I will keep using this after the assignment is graded.

## AI TOOLS AND APPS USED

| TOOL | ROLE |
|---|---|
| Claude (Opus 4.8) | Building and iterating the skill; running it against real sources |
| `skill-creator` | Anthropic's skill for making skills — asked clarifying questions, generated the SKILL.md |
| Claude.ai Skills | Installing the finished skill (Customize → Skills) |

## THE PROMPTS I USED, AND HOW I REFINED THEM

**Initial prompt to skill-creator:**

```
Use the skill-creator skill to build me a weekly study-notes skill.
Whenever I ask for "study notes" or "weekly notes", take my raw class
notes and format them as a heading per topic, a short bold key-terms
list, and exactly three review questions at the end. Ask me anything
you need, then build it.
```

**Clarifying questions it asked, and my answers:**

| QUESTION | MY ANSWER |
|---|---|
| Source of notes — pasted text or a file? | Raw text or a Markdown file |
| Do your notes already have headings? | No — invent them from the content |
| How many key terms per topic? | 5 |
| Plain review questions or MCQs? | MCQs |
| Language? | English only |

**Refinement (v1 → v2) — the prompt that mattered:**

```
I don't like the output. Make it an interactive HTML artifact instead —
eye-catching, so I don't get bored reading it. Modify the current skill.
```

This changed the output contract from *document* to *study tool*. Same content rules; the terms became tap-to-reveal and the answer key became a scored quiz. The v2 build section also pins a quality floor — no `localStorage` (it fails inside artifacts), real `<button>` elements, reduced-motion support — so the skill doesn't rediscover those constraints on every run.

## HOW I TESTED AND VERIFIED IT

| TEST | RESULT |
|---|---|
| **Content test**<br>Run against two real course sources | **Passed.** Skills & Connectors crash course → 11 cards; What AI Actually Is → 10 panels. Both correct: 5 tap-to-reveal terms each, exactly 3 scored MCQs. Checked card-by-card against the source that no facts were invented. |
| **Auto-trigger test**<br>Fresh chat, natural request, no slash command | **Outstanding.** Skill is installed; the run in a clean chat has not been done. |
| **Negative test**<br>Unrelated question in same chat | **Outstanding.** |
| **Description check**<br>"When would you use this skill, and when would you NOT?" | **Outstanding** as a clean test — see problem 2 below. |

## WHAT WORKED, WHAT DID NOT, AND PROBLEMS I FACED

**What worked**

- **The instructions port cleanly.** Given the same skill and two very different sources, the output format held exactly — same structure, same term count, same quiz length.
- **Deriving design from the source.** The skill tells the model to build the visual direction out of the notes' own metaphor. The Skills & Connectors notes use a kitchen analogy throughout, and the artifact came back as recipe cards in a card box. That instruction earns its place.
- **The failure that produced v2 was the useful part.** A skill can satisfy its own spec and still fail its purpose. Catching that required looking at the output

as a user, not as a checklist.

**What did not work**

- **The description is measurably too wide.** Three weak spots I can name: **(a)** "make notes" fires on meeting notes or a phone call, not just coursework; **(b)** "cleaned up or organized" overlaps the negative trigger — "clean up my notes" and "summarize my notes" are the same sentence to most people, so the positive and negative triggers disagree; **(c)** the artifact output is promised but never appears in the trigger logic, so someone wanting a quick reference gets a whole study tool.
- **The narrow description is a real cost.** v2 promises an HTML artifact, which means the skill will fight me if I ever want plain Markdown. Accepted deliberately — but it is a tradeoff, not a free win.

**Problems faced**

> **Problem 1 — I could not test triggering from inside the chat that built the skill.** The description is the only thing the model reads to decide relevance. But in the conversation where I built the skill, its full text was already loaded — so the model was following the instructions directly, not matching the description. Every "test" in that chat proves the instructions work and proves nothing about triggering. The two are separate claims and I initially conflated them.
>
> **Problem 2 — the same contamination broke my description check.** Asking "when would you use this skill?" is supposed to make the model paraphrase the description back so you can hear whether it's wider or narrower than you intended. Asked inside the build chat, the model was reading the body, the build instructions, and the design notes too. It flagged the three weak spots listed above — but that is an informed reading of the whole file, not the blind description match the test is designed to measure. **Both problems have the same fix: run it in a clean chat.**